

JD Cortex

Episodic Memory and Constraint-Driven Design in a Humanoid Robot Perception Stack

Bushra Ushaq · Tatheer Aima Naqvi

EZ-Robot JD Humanoid · Synthiam ARC · August 2026

github.com/aima0749/jd_cortex

Abstract. Most low-cost humanoid platforms perceive continuously and remember nothing: each frame is classified, acted on, and discarded, so the robot can describe what stands in front of it but cannot answer a question about five minutes ago. This report describes **JD Cortex**, a five-process perception, reasoning, and memory system for an EZ-Robot JD Humanoid driven by Synthiam ARC. Its distinguishing contribution is an episodic memory layer, a timestamped witness diary in SQLite, exposed to the language model as prompt *context* rather than as a separate query engine, so one call answers both “wave at me” and “who came in this morning” with no second path that can disagree. The dominant architectural forces here were platform constraints rather than algorithms, and we trace each to the decision it forced. We then present twelve fault case studies, three of which presented as application logic errors while originating in protocol handling, hardware state, and API metering.

Keywords. humanoid robotics; episodic memory; multi-process architecture; computer vision; language models; action whitelisting; constraint-driven design.

1 Introduction

A humanoid robot that can see is now cheap to build. Detectors are pre-trained, face recognition is a library call, and a language model will hold a conversation over HTTP. What remains hard is **persistence**: being asked about the past and answering from something actually recorded.

A perception-only robot is a classifier with a speaker attached. Asked “is anyone here?” it succeeds; asked “who was here an hour ago?” it either fails or, worse, confabulates, since a model given only the current scene will happily invent a fluent and fictional history. Closing that gap needs a recording substrate, a discipline about what is worth recording, and an integration strategy that keeps the recorded past and the observed present from contradicting each other.

JD Cortex does this on a deliberately constrained platform: a sixteen-servo bipedal robot on an EZ-B v4 board, driven by Synthiam ARC on its free tier. The platform is not the interesting part; the constraints are. One custom plugin slot. A camera not addressable by OpenCV. Two development machines with different accelerators. Roughly twenty model requests per day. Every significant decision traces to one of those four facts, and that traceability is more instructive than the architecture alone.

Contributions

1. **Episodic memory keyed on identity, not tracking.** Tracker identifiers are structurally unusable as a memory key; we replace them with recognised names plus one pooled *unknown* bucket, and debounce the two presence edges asymmetrically (§5).
2. **Single-brain integration of memory and conversation.** The diary is injected into the existing prompt as a labelled context block rather than routed to a query engine, so there is no second answering path to keep consistent (§6).
3. **Vocabulary without authority.** Every proposed action is validated twice against a hand-tested whitelist of 56 entries (§7).
4. **Twelve fault case studies**, each with its root cause and a lesson that carries to other systems — among them three bugs whose symptoms surfaced in a completely different part of the system from the fault itself (§9).
5. **Hardware-free verification:** 33 deterministic assertions, synthetic peripherals, and a staged bring-up ladder (§10).

2 Platform and Constraints

The robot is an EZ-Robot JD Humanoid with an EZ-B v4 controller, an onboard camera, and a chest speaker, over Wi-Fi. All actuation, speech synthesis, and camera access are mediated by Synthiam ARC, a Windows application exposing a scripting language and a TCP script server. ARC owns the hardware; nothing in this project drives a servo directly.

The two development machines differed in accelerator — one with an NVIDIA GPU, one CPU-only — which is why the inference device is an explicit setting rather than an assumption.

Table 1 lists the four constraints that shaped the system. We present them first because several later decisions are otherwise indistinguishable from arbitrary preference.

Table 1. Platform constraints and the decisions they forced

Constraint	Consequence
One custom plugin slot	The in-ARC panel is a thin renderer: it draws state and forwards one line of text. All decision logic stays in Python, because a richer plugin would consume the only slot available.
Camera not directly addressable	Frames come from ARC’s snapshot mechanism, and gesture control moved to a stationary webcam — for a reason stronger than convenience (§9.4).
Two machines, two accelerators	The inference device is an explicit setting checked against the driver at startup, and model size is a separate knob from it, so neither is chosen as a side effect of which laptop the pipeline happens to be on (§4).
≈ 20 model requests/day	Local literal matching before any network call; a three-key rotation ladder for exhaustion; ambient commentary separately disableable so conversation quota survives.

None of these was removed by spending effort; each was absorbed. The single-plugin limit in particular produced a cleaner separation than an unconstrained design would have: because the panel could not decide anything, the boundary between presentation and logic became physically enforced rather than merely intended.

3 Architecture

The system is five concurrent processes communicating through two files and two TCP ports (Fig. 1). Port 6666 is ARC’s script server; port 5005 is the panel server, which runs as a daemon thread *inside* the brain process rather than as a separate service, so panel input enters the same command path as typed and spoken input.

3.1 The scene file as an interface contract

The entire agreement between the perception half and everything downstream is one JSON file: a timestamp, the visible objects, and a dictionary of people — each with a name, a posture, what they are holding, and what they are sitting on. What it *omits* matters as much: no bounding boxes, no confidence scores, no tracker identifiers. Consumers receive conclusions, not detections, so the detection model can be replaced without any downstream change. Three rules govern it:

- **Single writer, mutually unaware readers.** Only the pipeline writes; four processes read; no reader knows about any other.
- **Freshness, not existence, is the validity test.** Anything older than ten seconds is treated as missing. A stale file is more dangerous than no file, because it is confidently wrong.
- **Missing is a valid state.** No file, stale file, and empty room all resolve the same way; the prompt omits that section. Nothing raises.

The file is written atomically — temporary path, then `os.replace` — so a reader never observes a partial write. The contract was fixed before either author wrote implementation code, and its main benefit was independence: the whole memory subsystem was built and tested against a hand-authored scene file, with no hardware present.

3.2 The one-speaker rule

Four processes can produce speech-worthy output, but only `main.py` holds an ARC connection for speaking; the others append to a file-backed queue that a background thread in the brain drains. The alternative fails specifically:

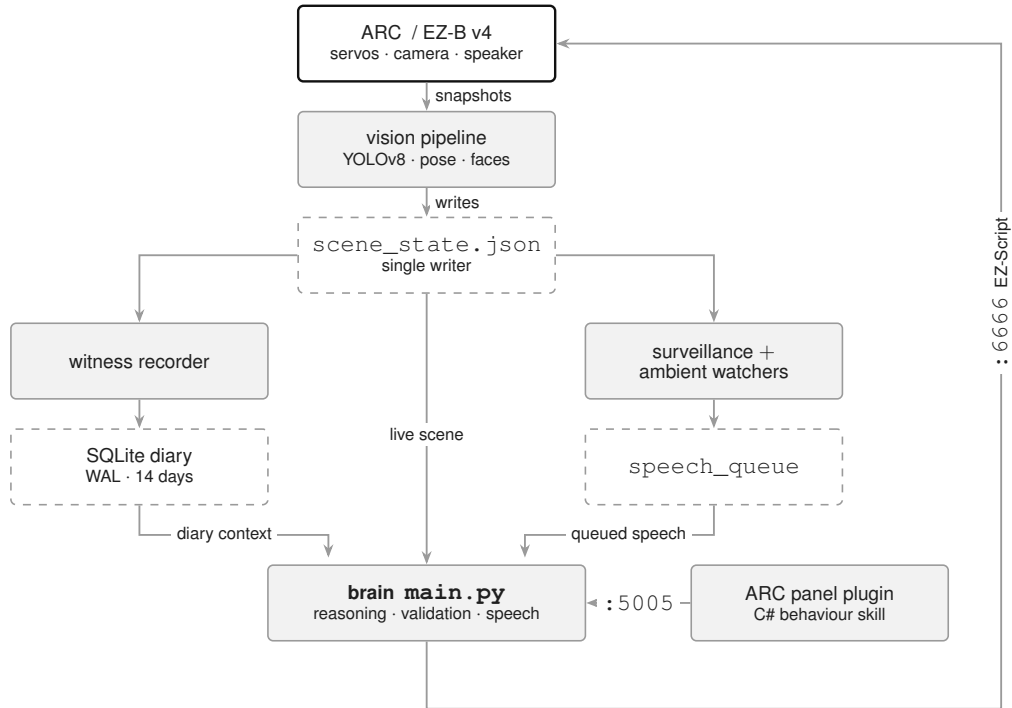


Figure 1. Process and data-flow topology. Shaded boxes are OS processes, dashed boxes are files acting as inter-process contracts, and the outlined box is the hardware owner. Perception flows down the centre; memory and autonomy branch left and right and converge on the brain, which is the only process that speaks or actuates.

two concurrent `SayEZBWait` calls interleave on one physical speaker, and the blocking semantics that make sequencing work are defined per connection. Centralising the speaker makes speech ordering a property of the system rather than a matter of timing luck.

4 Perception

The vision pipeline is a pure sensor process. It does not speak, actuate, or call the language model. It requests frames from ARC and runs three models per frame: **YOLOv8** over the 80-class COCO set for objects; **YOLOv8-pose** for per-person keypoints and track identifiers; and **dlib** 128-dimensional face embeddings matched by Euclidean distance against enrolled encodings. Embeddings were chosen over a trained classifier because enrolling a new person needs a handful of photographs rather than retraining.

Above these the pipeline derives the semantics that reach the scene file: whether a tracked person is holding a detected object (wrist-to-object proximity), whether they are seated, and a coarse posture estimate from change in bounding-box height. Those derivations are what let the scene file be a conclusions interface rather than a detections dump. Evidence capture uses a one-way channel: the surveillance watcher writes a request file, the pipeline writes an annotated frame and deletes the request. The watcher never touches the camera; the pipeline never decides what is worth photographing.

The pipeline runs on either accelerator, but which one is a setting, not a guess. It defaults to CUDA, checks the driver at startup, and if CUDA was expected and is absent it refuses to start and prints why rather than falling back silently — because on Windows a plain `pip install torch` installs the CPU-only build, and a silent fallback would hide that while the pipeline appeared to work and detected far less. CPU is opted into explicitly, and warns that it is a debugging mode rather than a quieter version of the same thing, since several confirmation and baseline constants are counted in frames. Model size is a separate knob from the device, deliberately: a smaller detector changes what the pipeline *detects*, and therefore the scene file and everything downstream, so it is never chosen as a side effect of which machine is running.

5 Episodic Memory: The Witness Diary

5.1 Why the naive design fails

The obvious way to give a robot a memory is to log what the tracker reports. The result, from this project’s own early activity log, is about thirty lines of:

person 14 arrived · person 14 left · person 15 arrived · person 15 left · person 16 arrived . . .

That is one person, over about one minute. Every time the subject turned or was briefly occluded, the tracker issued a fresh identifier and the logger faithfully recorded a stranger arriving.

This is not a tuning problem. A tracker identifier is a within-session construct with no commitment to persistence; using it as a memory key is a category error, and no threshold fixes a key that was never meant to be stable. The memory layer therefore ignores tracker identifiers entirely. Identity is a recognised name: each named person is one presence entry, and every unrecognised sighting collapses into a single *unknown* bucket carrying a count, rather than a growing crowd of imaginary strangers.

5.2 Asymmetric debouncing

Both presence edges are delayed before being believed, and deliberately by different amounts: an arrival needs **2.0 s** of continuous presence, a departure **12.0 s** of continuous absence. The asymmetry follows an asymmetry in the cost of error. Being late on an arrival costs two seconds and nothing else. Getting a departure wrong *corrupts the record*: it writes a departure time for someone who never moved, and every duration computed afterwards inherits the error. Twelve seconds survives an occlusion, a turn away from the camera, or a dropped frame.

Missing an event is cheap and recoverable; recording a false event is permanent. Debounce windows should be sized against that asymmetry, not against a single latency target.

Dropping a presence entry on departure also re-arms the arrival debounce, so a brief glimpse that never satisfied the window produces no event at all — which is the intent, not a gap.

5.3 Recording meaning, not frames

The recorder emits six event kinds and nothing else: *arrived*, *left* (with a humanised duration), *holding*, *sitting*, *object*, and *system*. Holding and sitting are recorded only for named, confirmed-present people, since attributing a held object to a pooled unknown is meaningless. A 300-second re-log gap stops a persistent object generating an event every cycle: the same cup in view for ten minutes is one event, not six hundred. The diary is readable because almost everything the camera sees is never written down.

Table 2. Diary schema, and the reasoning behind each storage decision

Column	Contents
<code>ts / kind</code>	event time (epoch); one of six event types
<code>person / detail</code>	a name, <i>unknown</i> , or <i>NULL</i> ; the object, count, or duration
<code>text</code>	the sentence the model actually reads
Decision	Rationale
WAL journal mode	The recorder writes while the brain reads from another process and neither blocks, with a 5 s busy timeout as backstop.
Per-operation connections	At a few events per minute the connect cost is nothing, and a process that dies mid-operation cannot hold the database hostage.
Indexes on <code>ts</code> , (<code>person</code> , <code>ts</code>)	The two questions actually asked are “what happened this morning” and “when was this person last here.”
14-day retention	Pruned at recorder start. Half performance, half privacy (§11.2).

The `text` column is worth singling out: the recorder writes the human-readable sentence at record time rather than rebuilding it at query time. The read path stays trivial (retrieval is a concatenation, not a templating exercise) and the phrasing the model sees was chosen by code that still had full context. The recorder never opens its own ARC connection; with arrival announcements enabled it writes to the same speech queue the watchers use, so the one-speaker rule holds everywhere.

6 Reasoning: A Single Brain

The conventional way to add memory to a conversational robot is a second mode: detect a memory question, route it to a query engine, have that engine answer. We deliberately did not build that. A second answering path is a second thing that can disagree with the first, and reconciling the two is a permanent tax.

Instead the diary is *context*. It enters the same prompt as its own labelled block, alongside the live scene and the action list, so the same call that handles “wave at me” handles “who came in this morning.” Adding memory therefore required no change to the command loop: the diary block is read fresh per command and treated as optional, exactly like the scene block. The two blocks stay explicitly distinct, because merging them makes the model report as present someone who left an hour ago — labelling turns the temporal distinction into an instruction it must respect rather than something it infers from ordering.

6.1 Contract and fallback

The model is not the front door. Every input is matched literally against the known-action list first, and only unmatched input reaches the network. That single ordering buys lower latency on the common path, conservation of a scarce daily quota, and offline operation for frequent commands at once.

When a call is made, the reply must be exactly two labelled lines — `REPLY`: the sentence spoken, and `MATCH`: a category (`movement`, `sound`, `light`, or `NONE`) and a name. The rigidity is the engineering point: asking a model for structured data in its own format fails often enough to lose a live demonstration, while two labelled lines effectively never do and parsing them is a string split. A missing or unparseable `MATCH` is treated as `NONE`.

Failure defaults to inaction. Nothing in this system turns an unparseable reply into a servo movement.

Beneath the contract sits a fallback ladder whose rungs differ by error semantics rather than uniform retry. A **404** means the model name is dead, so the next model in the candidate list is tried. A **timeout** is transient, so it retries twice, 1.5 s apart, with a 20 s per-attempt timeout. A **429** means quota is exhausted, which retrying cannot fix, so the next of up to three API keys is used. A uniform “retry three times” policy would have been simpler and strictly worse: it converts a permanent failure into a slower permanent failure while consuming the quota it is trying to preserve.

7 Safety and Autonomy

The model suggests actions; it does not execute them. What returns from the network is text, checked against a whitelist written and physically tested by hand: **33** Auto Position movements, **10** soundboard tracks, **13** lighting effects. Validation happens twice — when the reply is parsed, and again immediately before dispatch — so nothing can change in between, and after a movement the robot returns to standing. If the model invents an action, nothing happens and the robot says it cannot do that. The whitelist is not merely a text filter: every entry is a movement built and physically executed on the robot, so the worst case is not an arbitrary servo position but a wrong yet already-safe gesture.

The model contributes vocabulary. It does not contribute authority. There is no code path from an invented sentence to a servo angle.

The two autonomous behaviours are built to deliberately different reliability standards (Table 3). The surveillance watcher is offline by design, because a safety alert that needs a working internet connection is not a safety alert; its sensitive-object list is scoped to unrecognised people, since a known occupant holding scissors is not an incident and a stranger holding them may be. The ambient watcher is the opposite — allowed to need the network, to spend quota, and to go quiet when either runs out. Neither speaks directly; both append to the queue the brain drains.

8 Human Interfaces

The panel is a C# behaviour skill running inside ARC, so an operator never looks at a terminal. It shows a connection indicator, a live card of the current scene, the day’s security log, a counter for model calls consumed, question chips that fill in the currently visible person’s name, hold-to-talk, and a gesture toggle.

Its protocol is deliberately minimal: one line in, one line out, on localhost port 5005, with a reserved vocabulary of

Table 3. Two watchers, two reliability contracts

	Surveillance	Ambient
Purpose	Safety alerting	Companionship
Network	None — fully offline	Requires model access
Confirmation	3 consecutive polls	Change-triggered
Cooldowns	120 s unknown person, 60 s sensitive object	Model decides whether to speak at all
Night window	22:00–06:00 alters thresholds	—
Scoping	Object alerts fire only for unrecognised people	—
On failure	Must still work	Allowed to go silent

ping, status, diary, alerts, gesture, listen, and stop. Anything outside that closed set is free text for the brain. The closed set resolves a real ambiguity — if a user types “status” meaning it as a question, it must not be swallowed as a protocol command — and §9.6 shows what getting this wrong cost. Because the server is a daemon thread inside the brain’s own process, free text enters the identical function as typed and spoken input, with the same connection and the same validation.

Transcription runs locally through a Parakeet model via sherpa-onnx; no audio leaves the machine. Input is push-to-talk, and the module checks a speaking flag first so the robot never transcribes itself.

Gesture control uses MediaPipe Hands (21 landmarks per frame) with a geometric classifier mapping normalised configurations to eight commands (Table 4). A gesture must hold for five consecutive frames before it commits, removing transitional misfires as the hand changes shape; if the hand disappears for fifteen frames while a motion command is active, a stop is issued unconditionally. The vocabulary is sized to an input/output impedance match — landmark detection distinguishes many finger configurations, but the robot expresses commands as gross-motor servo actions — so separable hand poses are paired with unambiguous whole-body movements rather than maximising the gesture count. Frames come from a stationary laptop webcam, never the robot’s own camera; the reason is §9.4, and it is not convenience.

Table 4. Gesture vocabulary

Hand shape	Action
Fist / Open hand	Walk forward / Stop
Index pointing left / right	Turn left / right
Index pointing down	Sit down
Peace sign / Three fingers	Wave / Stand from sit
Index + pinky	Push-ups
<i>Hand leaves frame</i>	<i>Safety stop, after 15 frames</i>

9 Fault Case Studies

These twelve faults are recorded not as a defect log but because several share a property worth generalising: **the symptom appeared at a layer well above the cause**. Three presented as application logic errors while originating in protocol handling, hardware state, and API metering respectively.

9.1. The greeting banner off-by-one

Speech overlapped and sentences were truncated: the blocking call appeared to return before the sentence finished. ARC’s script server sends a greeting banner on every new connection, and the client never read it — so the banner was consumed as the reply to the first command, the first reply as the reply to the second, and so on for the whole session. Fixed by draining the socket to the prompt character on connect.

└ When request/response pairing is systematically shifted rather than intermittently wrong, suspect the transport, not the logic.

9.2. The mute speaker illusion

Every movement executed and no sound came out, with scripting, TCP, and the board link demonstrably fine. Two faults compounded. The speech call had been targeting the host PC’s speakers, so early testing sounded correct

while proving nothing; correcting it produced total silence, which looked like a regression. Underneath, the board's volume (independent of Windows volume) was at zero from a previous session, so the correct call was running on a muted device.

┆ *A correct call with no observable effect is a hardware-state hypothesis. And a fix that appears to make things worse may be landing on a second, previously masked fault.*

9.3. The phantom quota exhaustion

Conversation succeeded while memory questions failed with quota errors — same key, same session, minutes apart. The first conclusion, “the key is exhausted,” was wrong, and was disproved by the operator noticing the failure was inconsistent. The two halves were calling different endpoints: one native, one a compatibility layer metered separately and far more tightly. Fixed by consolidating to a single native client, then adding model fallback and three-key rotation.

┆ *Inconsistent failure is evidence against a global cause; the difference between the component that fails and the one that does not is the diagnosis.*

9.4. The self-cancelling gesture loop

The forward gesture made the robot take one step and stop. Gesture frames originally came from the robot's own camera, and the safety rule stops motion when the hand is lost. But the camera is mounted on the robot: forward makes it walk, walking shakes the view, the operator leaves frame, the hand is lost, and the stop fires. The one gesture whose purpose is motion was the one guaranteed to cancel itself — a closed feedback loop created by sensor placement, not a tuning failure. Fixed by sourcing frames from a stationary webcam.

┆ *When a control loop's sensor is mounted on its own actuator, check for self-cancellation before tuning anything. Some designs are not fragile, they are impossible.*

9.5. Silent loss of an entire action class

Sound requests produced a spoken acknowledgement and no sound — no error, no traceback, no log line. The prompt described available sounds by their display labels, so the model faithfully returned `sound|Track 9: Happy Birthday.mp3`; the parser expected a bare integer, conversion failed, validation matched nothing, and the action was discarded. Instrumentation showed dispatch was never reached. Fixed by extracting the leading integer from the label before validation.

┆ *The prompt taught the model the wrong format — check what you showed it before assuming model error. And this had been killing every sound request, not just the one that surfaced it. Silent discard generates no evidence, which makes it the most expensive failure mode here.*

9.6. Panel command fallback

Daily quota drained within minutes of connecting the panel, and the robot performed actions nobody asked for. The panel polls for status; an unrecognised polling command fell through the reserved-word check and was forwarded to the model as free text roughly every two seconds. Each poll spent quota, and each reply could carry an action match that then executed. Fixed by closing the reserved vocabulary and implementing the missing commands.

┆ *Fallback on undefined protocol input is safe only when the fallback path is cheap and side-effect free. Here it was metered and it was actuating.*

9.7. Tracker identifiers crossing the user boundary

The robot spoke sentences like “person 31 is standing,” and passed the same phrasing into prompts: the live scene summariser fell back to the tracker identifier when no name was available. This is the diary problem's category error surfacing at a different layer, and it made one person sound like several as the identifier churned.

┆ *Internal identifiers must be stripped at the boundary, not merely discouraged. Fixing the memory layer did not fix the live layer, because each had its own tempting default.*

9.8. A latent merge hazard

No symptom, caught during merge review, before it ran. Independent work on two branches produced a variable-name collision in the reasoning module: a local variable from one branch would have shadowed the diary parameter from the other. Git reported no conflict, because the edits were in different regions of the file, so the merge would have been clean and the memory context silently empty.

┆ *A clean merge is a syntactic result, not a semantic one. When two branches touch the same function for different reasons, review the merged function, not the diff.*

9.9. Line-ending noise in review

`git diff` reported far more modified files than had actually been touched, making review impractical. Git for Windows was converting line endings on checkout, so untouched files appeared fully rewritten.

┆ *When the volume of change does not match the work performed, suspect the tooling before auditing your own edits.*

9.10. A removed API, not a deprecated one

The gesture library installed successfully and the required namespace did not exist at runtime. Newer MediaPipe releases removed the solutions API this module uses — no deprecation warning, because the feature was gone rather than deprecated. Fixed by pinning to a verified compatible pair and documenting the pin, with its reason, in the dependency file.

┆ *A pin without a recorded reason gets removed by the next person tidying dependencies. The comment is part of the fix.*

9.11. A silently dropped process

The unified launcher started, reported success, and the vision pipeline simply was not running, although it ran correctly when started by hand. On the CPU-only machine the pipeline found no CUDA device and refused to start, exactly as §4 describes — but that diagnostic is only useful if somebody reads it. Started directly it was visible in the pipeline’s own console; under the launcher it went to a window that closed, and liveness was never verified after spawn.

┆ *A supervisor that spawns without verifying is worse than none, because it turns a loud failure into a silent one — the same anti-pattern as 9.5, in process management rather than parsing.*

9.12. A duplicated safety list

Two copies of the action whitelist exist in different directories and have drifted. An early refactor left the shared copy behind; nothing imports it, so it is unreachable, but its existence invites future edits to the wrong file. Verified still present while writing this report: the two copies differ by one movement entry.

┆ *Recorded rather than elided, because a duplicated safety list is a latent safety defect, not untidiness.*

10 Verification Without Hardware

Hardware access was intermittent and the robot was shared, so the strategy was to push as much confidence as possible into environments needing neither the robot nor ARC.

Both memory components carry executable self-tests that run with no camera, robot, ARC, or network, against a throwaway database and synthetic landmark geometry. Re-executed against the current `main` for this report, `witness_recorder.py selftest` gives **20 passed, 0 failed** — presence grouping, unknown pooling, both debounce edges, duration humanisation, re-log suppression, store round-tripping, context size capping — and `gesture_control.py selftest` gives **13 passed, 0 failed**, covering every classification, commit debouncing, the safety stop on hand loss while moving, the correct *absence* of a stop while stationary, and completeness of the command-to-action mapping.

These 33 assertions substitute for a conventional test suite, chosen to cover the logic hardware testing exercises least reliably: timing-dependent state machines, hard to trigger deliberately in front of a robot. Two test doubles supported them — a synthetic scene generator and a mock ARC endpoint, against which the §9.1 banner fix was validated.

Table 5. Hardware bring-up ladder — each rung establishes a precondition for the next

Rung	What it proves
1. Chest speaker, three-line ordering	Audio path and blocking semantics
2. Auto Position actions	TCP, scripting, and the board link
3. Vision pipeline start-up	Models, device resolution, scene file production
4. Diary chain and panel query	First run of camera → diary → model → speaker
5. Gesture control, clear floor	Stop tested <i>before</i> forward
6. Two timed full rehearsals	Ordering faults and dead air

Rung 5 encodes a convention worth stating: *test the brake before the accelerator*. The ladder also paid off directly — when rung 2 passed and rung 1 failed, the fault localised to audio rather than connectivity, which is what led to the

diagnosis in §9.2.

11 Engineering Practice

11.1 Ownership and branches

The two contributors had explicitly separated ownership: perception, reasoning, action execution, and watchers on one side; memory, the ARC panel and its server, gesture control, and the launcher on the other, with cross-boundary edits kept surgical by policy. With the scene-file interface agreed in advance, the memory subsystem reached completion before the two halves were ever run together.

Development ran on three branches: `main`, a `memory-feature` branch carrying the memory subsystem, and a small `pipeline-device-flag` branch for explicit device selection. Significant work — the tracker-identifier fix, a large panel-server revision, and a rewrite of the plugin’s main form — stayed on the feature branch and never reached `main`, until an audit found it. Had a demonstration run from `main`, none of it would have been present, and the failure would have looked like regression rather than work never merged. The merge itself was clean, so the lesson is not about merge mechanics: *verify which commit the demonstration actually runs from*, because a branch is not a deliverable.

A documentation rewrite on `main` separately removed every mention of the memory subsystem, the plugin, the panel, and gesture control. Documentation regressions are invisible to every automated check: they are found only by reading, and found late.

Behaviour is configured entirely through environment variables — device and model selection, speech target and volume, API keys, diary path and retention, gesture camera index, announcement mode — and the launcher resolves the repository from its own location, starts the five processes in dependency order, and terminates them together. Its `check` subcommand audits libraries, model files, keys, and ports before a session is committed to, because the most expensive failures here were silent absences rather than crashes.

11.2 Privacy and data governance

A robot that logs the movements of identified people is an ethics question before it is an engineering one. The diary keeps 14 days and prunes older events at startup — enforced in code, not documented as policy. Four classes are deliberately untracked: the database, enrolled photographs, computed encodings, and captured snapshots. Transcription is on-device. And sensitive-object alerts fire only for unrecognised people, limiting routine surveillance of known occupants.

The audit that found the stranded branch also found a captured snapshot of an unidentified person committed to the public repository. It was removed and the ignore rules corrected. That is recorded rather than quietly fixed, because the policy was correct and the enforcement had a gap, which is the ordinary way privacy failures happen.

A stated retention policy is not protection. The ignore rules and the pruning code are.

12 Results, Limitations, and Future Work

Table 6. System parameters, as configured in the current `main`

Property	Value
Concurrent processes	5
Inter-process contracts	2 files, 2 TCP ports
Whitelisted actions (movement / sound / light)	33 / 10 / 13
Gesture commands / diary event kinds	8 / 6
Presence debounce (arrival / departure)	2.0 s / 12.0 s
Scene freshness window / diary retention	10 s / 14 days
Self-test assertions	33

We deliberately report no latency or accuracy figures: no instrumented measurement campaign was run, and quoting impressions as measurements would misrepresent the verification actually performed. Functional verification on

hardware covers speech through the onboard speaker with correct blocking semantics; the whitelisted action set; the vision pipeline on both accelerators; the full camera-to-diary-to-answer chain; panel connection, live scene card, and query commands; hold-to-talk transcription; and gesture control including the safety stop.

Open items, stated plainly because a report listing only what works is less useful than one that says where the edges are. The launcher does not pass the device setting through, so on a machine without CUDA it can still drop the vision process without saying so — a one-line fix, deferred to avoid touching the launcher before a demonstration. Several files are unreferenced by any import, including the drifted whitelist of §9.12. The plugin binary needs rebuilding after the merge of its main form. Only one contributor is enrolled for face recognition, and there is no continuous integration.

Design limitations. The system has no depth sensing, which is why autonomous navigation to a named object is not implemented: one camera gives bearing but not range, so the robot knows a target’s direction but not its distance, and cannot detect a table edge. The intended remedy is a small sensing layer — ultrasonic ranging forward, infrared for ledge detection — on a dedicated microcontroller rather than the robot’s own board, since the servo power rails carry substantial electrical noise and the logic voltages differ.

Directions. Enrolment currently needs pre-supplied photographs; a designed extension lets an unrecognised person enrol conversationally, spelling their name letter-at-a-time, since the alphabet is a closed vocabulary where a non-English name is not. The more valuable extension is relevance-based retrieval over the diary: the current design injects a recent window with a size cap, and as retention grows that becomes the binding constraint on memory quality. Query-conditioned selection would raise that ceiling while keeping the diary context.

13 Conclusion

The interesting difficulty in building a memory-capable humanoid on commodity components is not model selection. It is deciding what deserves to be remembered, structuring memory so it cannot contradict live perception, and absorbing platform constraints rather than fighting them.

Three results generalise beyond this platform. **Episodic memory must be keyed on semantic identity, not tracking state**; per-frame identifiers are the wrong type for the job, and the error recurs at every layer that has one available as a fallback. **Memory should be context, not a second answering path**; injecting the record into the existing call eliminates a whole class of consistency bug for the cost of a prompt block. And **failure should be silent nowhere**; the most expensive faults here — a discarded action class, a dropped process, a stranded branch — all produced no evidence, and each was found by audit rather than observation.

The case studies support a narrower claim we would defend as strongly: in a system spanning a robot, proprietary middleware, four inference models, and a metered API, the layer at which a symptom appears is weak evidence about where it originates.

Contributions:

Bushra Ushaq — SQLite witness diary and recorder, ARC Memory panel (C# skill and Python server), MediaPipe gesture control, unified launcher.

Tatheer Aima Naqvi — YOLO vision pipeline, Gemini conversation and intent matching, whitelisted action execution, unknown-face and sensitive-object surveillance, ambient mode.

Availability:

Source is at https://github.com/aima0749/jd_cortex — source only; model weights, enrolled photographs, encodings, the diary database, and snapshots are excluded. Both memory components run with no hardware attached via their selftest entry points.

References:

[1] Synthiam ARC robot control software, synthiam.com. [2] G. Jocher et al., “Ultralytics YOLOv8,” 2023. [3] D. E. King, “Dlib-ml,” *JMLR*, vol. 10, pp. 1755–1758, 2009. [4] A. Geitgey, “face_recognition,” github.com/ageitgey/face_recognition. [5] C. Lugaresi et al., “MediaPipe,” arXiv:1906.08172, 2019. [6] F. Zhang et al., “MediaPipe Hands: On-device real-time hand tracking,” arXiv:2006.10214, 2020. [7] Next-gen Kaldi, “sherpa-onnx,” github.com/k2-fsa/sherpa-onnx. [8] SQLite, “Write-Ahead Logging,” sqlite.org/wal.html. [9] T.-Y. Lin et al., “Microsoft COCO,” *ECCV*, 2014, pp. 740–755.